

Topic 2.5

2.5: Multi-file Programs

It is often the case that large programs consist of several C source files. Usually, each C source file will have an associated ".h" header file that contains prototypes of the functions defined in each source file, constants used by the source file, etc.

Splitting C sources across files is useful for several reasons. Doing this

- helps to organize functionality,
- provides different scopes and extents for variables and functions,
- allows multiple programmers to work on the same code in an effective manner, and
- enables programmers to reuse code.

Compiling and loading an application spanning multiple files is system dependent. On UNIX systems,

```
cc -o exec_name *.c
```

compiles all C source files and links them, creating an executable with the name *exec_name*.

The following example C program illustrates the use of projects and multi-file programs:

```
/*
 * FILENAME:   what.c
 * PROGRAMMER: Gary J. Blair
 * DATE:       7/13/1992
 * LANGUAGE:   ANSI C
 * PURPOSE:    Illustrate the use of projects/multi-file programs.
 * NOTES:      This code resides in file what.c.
 */

#include <stdio.h>
#include "f.h"

int main(void)
{
    float x, y;

    printf("Enter a float value for x ");
    scanf("%f",&x);
    y = f(x);
    printf("f(%g) = %g\n",x,y);
    return 0;
}

/*
 * FILENAME:   f.h
 * PROGRAMMER: Gary J. Blair
 * DATE:       7/13/1992
 * LANGUAGE:   ANSI C
 * PURPOSE:    Illustrate the use of projects/multi-file
 *             programs.
 *
 * NOTES:
 */
```

```

*   This code resides in file f.h.
*/

extern float f( float x );

/*
* FILENAME:      f
* PROGRAMMER:    Gary J. Blair
* DATE:          7/13/1992
* LANGUAGE:      ANSI C
* PURPOSE:
*   Illustrate the use of projects/multi-file programs.
*
* NOTES:
*   This code resides in file f.c.
*/

#include "f.h"

floatf( float x )
{
    return x * x - 2 * x + 17;
}

```

Compiling and Linking a Multi-file Program

First, we need to compile the individual source files. The following commands illustrate how to do this on a UNIX system:

`cc -c what.c`

creates the object file *what.o*.

`cc -c f.c`

creates the object file *f.o*.

Then it is necessary to link the object files together. The linker may be run directly, or we can use the compiler `cc` to run the linker for us:

`cc -o what what.o f.o`

creates the executable file *what*.

Finally, we execute the program:

```

% what
Enter a float value for x 234.5
f(234.5) = 54538.2

```

Topic 2.6 (<https://onlinelearning.berkeley.edu/courses/2291660/pages/topic-2-dot-6>)